

BASH SCRIPTING

Complete Exam Notes

Theory + Syntax + Examples + Outputs + Ready-made Exam Programs

No.	Chapter
1	Introduction, Shebang, Running Scripts
2	Variables, Special Variables, Quoting
3	Input and Output (echo, printf, read)
4	Arithmetic
5	Test Conditions and Operators
6	Conditionals: if, elif, case
7	LOOPS: for, while, until, select, break, continue
8	Arrays
9	Functions and Recursion
10	String Manipulation
11	Redirection, Pipes, Here Documents
12	Command-line Arguments, shift, getopts
13	Exit Status, trap, Debugging, set options
14	Useful Commands: grep, sed, awk, find ...
15	File Handling and Process Control
16	Exam Programs (20 solved programs)
17	Viva / Short Questions and Answers
18	Quick Cheat Sheet and macOS Notes

Important for macOS: Everything here works in bash. macOS ships an old bash 3.2, so associative arrays and case-conversion like `${x^^}` need bash 4+ (install with `brew install bash`). These are marked with [bash 4+] in the notes.

1. Introduction to Shell and Bash

1.1 What is a Shell?

A **shell** is a command-line interpreter. It takes commands typed by the user (or written in a file), interprets them and asks the **kernel** (the core of the OS) to execute them. It works as the interface between the user and the operating system.

Shell	Full form / Notes
sh	Bourne Shell - original Unix shell (Stephen Bourne)
bash	Bourne Again SHell - GNU shell, default on most Linux distros
csh / tcsh	C Shell - C-like syntax
ksh	Korn Shell (David Korn)
zsh	Z Shell - default on macOS since Catalina; mostly bash compatible

1.2 What is a Shell Script?

A **shell script** is a text file containing a sequence of shell commands, executed line by line by the interpreter. It is used for automation, system administration, batch jobs, backups and installation tasks.

Advantages

- Automates repetitive tasks; saves time.
- Easy to write and debug (no compilation).
- Portable across Unix-like systems.
- Can combine any Unix command using pipes and redirection.

Disadvantages

- Slower than compiled languages (C, Java).
- Weak data structures; not ideal for heavy computation.
- Syntax is strict about spaces and quoting; error-prone.

1.3 Shebang (#!)

The first line `#!/bin/bash` is called the **shebang** (or hashbang). It tells the OS which interpreter must run the script. `#!/usr/bin/env bash` is more portable because it finds bash from the PATH.

1.4 Creating and Running a Script

```
# Step 1: create the file
nano hello.sh

# Step 2: content of hello.sh
#!/bin/bash
# This is a comment
echo "Hello, World!"

# Step 3: give execute permission
chmod +x hello.sh

# Step 4: run it (any of these)
./hello.sh      # needs execute permission + shebang
bash hello.sh   # no execute permission needed
sh hello.sh
source hello.sh  # or: . hello.sh (runs in CURRENT shell)
```

Output

```
Hello, World!
```

Comments

```
# Single-line comment
echo "Hi" # inline comment

: '
This is a
multi-line comment
'

# Another trick with a here-document
<<COMMENT
Anything here is ignored
COMMENT
```

Exam Tip: `./script.sh` runs in a child shell; `source script.sh` runs in the current shell so variables it sets remain after it finishes. `chmod +x` gives execute permission (mode 755 = `rwxr-xr-x`).

2. Variables

2.1 Rules for Variables

- Name may contain letters, digits and underscore; **cannot start with a digit**.
- Names are **case sensitive**: `NAME` and `name` are different.
- **No spaces around =** while assigning: `x=10` is right, `x = 10` is wrong.
- Use `$` to read a variable: `$x` or `${x}`. Do not use `$` when assigning.
- By default every value is a string; bash converts to number in arithmetic context.

```
name="Hamim" # string
age=21 # number (stored as string)
echo "Name: $name, Age: $age"
echo "Name: ${name}123" # braces separate the variable name from text
readonly PI=3.14 # constant, cannot be changed
unset age # delete a variable
declare -i count=5 # integer attribute
declare -r MAX=100 # readonly
declare -x MYVAR=hello # export (same as: export MYVAR=hello)
```

Output

```
Name: Hamim, Age: 21
Name: Hamim123
```

2.2 Types of Variables

Type	Meaning / Example
Local (user-defined)	Created by the user in the script: <code>x=5</code>
Environment	Available to child processes after <code>export</code> : <code>export PATH=\$PATH:/opt/bin</code> . Examples: HOME, USER, PATH, PWD, SHELL, HOSTNAME
Special / positional	Set by the shell: <code>\$0 \$1 \$# \$? \$\$...</code> (see next table)

2.3 Special Variables

Variable	Meaning
<code>\$0</code>	Name of the script
<code>\$1 ... \$9, \${10}</code>	Positional parameters (command-line arguments)
<code>\$#</code>	Number of arguments
<code>\$*</code>	All arguments as ONE single string ("a b c")
<code>\$@</code>	All arguments as SEPARATE words ("a" "b" "c"); use <code>"\$@"</code>
<code>\$?</code>	Exit status of the last command (0 = success, non-zero = failure)

Variable	Meaning
<code>\$\$</code>	PID (process ID) of the current shell/script
<code>\$!</code>	PID of the last background process
<code>\$-</code>	Current shell option flags
<code>\$_</code>	Last argument of the previous command

```
#!/bin/bash
echo "Script name : $0"
echo "First arg   : $1"
echo "Second arg  : $2"
echo "Total args  : $#"
```

Output

```
$ ./args.sh apple banana
Script name : ./args.sh
First arg   : apple
Second arg  : banana
Total args  : 2
All args    : apple banana
PID         : 4821
Last status : 1
```

2.4 Quoting

Quote	Behaviour	Example
Double <code>" "</code>	Allows <code>\$var</code> , <code>\$(cmd)</code> , backslash escapes; prevents word splitting	<code>echo "Hi \$USER" -> Hi hamim</code>
Single <code>' '</code>	Everything literal, NO expansion	<code>echo 'Hi \$USER' -> Hi \$USER</code>
Backslash <code>\</code>	Escapes ONE next character	<code>echo \ \$HOME -> \$HOME</code>
Backtick	Old style command substitution (use <code>\$()</code> instead)	<code>d=date`</code>

2.5 Command Substitution

```
today=$(date +%F)           # modern form (preferred, can be nested)
files=`ls | wc -l`          # old form
echo "Today is $today and there are $files files"
echo "Kernel: $(uname -r)"
```

2.6 Environment Variables and export

```
x=10                        # local: not visible to child processes
export y=20                 # exported: child processes can see it
bash -c 'echo "x=$x y=$y"'
echo $PATH                  # list of directories searched for commands
env                          # print all environment variables
printenv HOME               # print one
```

Output

```
x= y=20
```

3. Input and Output

3.1 echo

Option / escape	Meaning
-n	Do not print trailing newline
-e	Enable backslash escapes: \n newline, \t tab, \\ backslash, \a beep
-E	Disable escapes (default)

```
echo "Hello"
echo -n "No newline "; echo "same line"
echo -e "Line1\nLine2\tTabbed"
```

Output

```
Hello
No newline same line
Line1
Line2■Tabbed
```

3.2 printf (formatted output like C)

Format	Meaning
%s	string
%d	integer
%f	float
%5.2f	width 5, 2 decimals
%-10s	left-aligned in 10 columns
%05d	zero-padded to 5 digits
%x %o	hex, octal
\n	newline (not automatic!)

```
printf "%s is %d years old\n" "Hamim" 21
printf "%-10s|%5d|%8.2f\n" "Apple" 5 12.5
printf "%05d\n" 42
```

Output

```
Hamim is 21 years old
Apple      |    5|   12.50
00042
```

3.3 read (input)

Option	Meaning
-p "prompt"	Show a prompt
-s	Silent (password entry)
-n N	Read only N characters
-t N	Timeout after N seconds
-a arr	Read words into an array
-r	Raw: do not treat backslash as escape
-d ch	Use ch as delimiter instead of newline

```
read name                # simple
read -p "Enter your name: " name  # with prompt
```

```

read -s -p "Password: " pass; echo      # hidden input
read -n 1 -p "Continue? (y/n) " ans     # one key
read -t 5 -p "Quick! " x                # 5 second timeout
read -a nums <<< "10 20 30"             # read into array
read a b c <<< "one two three four"     # c gets "three four"
echo "Hello, $name"

```

Exam Tip: If there are fewer variables than words, the LAST variable takes the remaining words. With no variable name, input goes into the built-in variable **\$REPLY**.

4. Arithmetic in Bash

Bash performs **integer** arithmetic only. For decimals use **bc** or **awk**.

```

a=10; b=3
echo $((a + b))      # 13   arithmetic expansion (best)
echo $((a * b))      # 30
echo $((a / b))      # 3    integer division
echo $((a % b))      # 1    modulus
echo $((a ** b))     # 1000 exponent
c=$((a + b))         # store the result
(( a++ ))            # increment (also ++a, a--, --a)
(( a += 5 ))         # compound assignment
let "d = a * 2"      # let command
e=$(expr $a + $b)    # expr (old way; spaces required, escape * as \*)
echo "scale=2; 10/3" | bc      # 3.33 floating point with bc
echo "scale=4; sqrt(2)" | bc -l # 1.4142
awk 'BEGIN{print 10/3}'        # 3.33333

```

Operator	Meaning	Operator	Meaning
+ - * /	add, subtract, multiply, divide	%	modulus
**	power	++ --	increment, decrement
+= -= *= /= %=	compound assignment	& ^ ~	bitwise AND, OR, XOR, NOT
<< >>	shift left / right	&& !	logical AND, OR, NOT (in (()))

Exam Tip: Inside **\$(())** and **(())** you do not need **\$** before variable names. **(())** returns true (exit 0) when the result is non-zero - so it can be used directly in **if**.

5. Test Conditions and Operators

Conditions are evaluated by **test**, **[]** (same as test), **[[]]** (bash extended) or **(())** (arithmetic). There **must be a space** after **[** and before **]**.

5.1 Numeric Comparison

Operator	Meaning	In (())
-eq	equal	==
-ne	not equal	!=
-gt	greater than	>
-lt	less than	<
-ge	greater or equal	>=
-le	less or equal	<=

5.2 String Comparison

Operator	Meaning
<code>=</code> or <code>==</code>	strings are equal
<code>!=</code>	strings are not equal
<code>-z str</code>	string is empty (zero length)
<code>-n str</code>	string is NOT empty
<code><</code> <code>></code>	lexical (dictionary) order, inside <code>[[]]</code>
<code>[[\$s == a*]]</code>	pattern (glob) match inside <code>[[]]</code>
<code>[[\$s =~ ^[0-9]+\$]]</code>	regular expression match

5.3 File Test Operators

Operator	True if ...
<code>-e f</code>	file/directory exists
<code>-f f</code>	exists and is a regular file
<code>-d f</code>	exists and is a directory
<code>-r f -w f -x f</code>	readable / writable / executable
<code>-s f</code>	exists and size is greater than zero
<code>-L f</code>	is a symbolic link
<code>f1 -nt f2</code>	f1 is newer than f2
<code>f1 -ot f2</code>	f1 is older than f2

5.4 Logical Operators

Operator	Use
<code>!</code>	NOT
<code>-a</code> or <code>&&</code>	AND (<code>[a -a b]</code> or <code>[[a && b]]</code>)
<code>-o</code> or <code> </code>	OR

5.5 `[ ]` versus `[[ ]]`

<code>[]</code> (test)	<code>[[]]</code> (bash keyword)
POSIX, works in sh	Bash/ksh/zsh only
Must quote variables: <code>["\$x" = a]</code>	Quoting optional; no word splitting
Use <code>-a</code> <code>-o</code> for AND/OR	Use <code>&&</code> <code> </code> inside
No pattern or regex	Supports <code>==</code> glob and <code> =~ </code> regex

6. Conditional Statements

6.1 if - then - fi

Syntax

```
if [ condition ]
then
    commands
fi

n=15
if [ $n -gt 10 ]; then
    echo "$n is greater than 10"
fi
```

6.2 if - else

```
read -p "Enter age: " age
if [ $age -ge 18 ]; then
    echo "Adult"
else
    echo "Minor"
fi
```

6.3 if - elif - else (ladder)

```
read -p "Enter marks: " m
if [ $m -ge 80 ]; then echo "A+"
elif [ $m -ge 70 ]; then echo "A"
elif [ $m -ge 60 ]; then echo "A-"
elif [ $m -ge 50 ]; then echo "B"
else
    echo "Fail"
fi
```

6.4 Nested if

```
read -p "Enter a number: " n
if [ $n -ge 0 ]; then
    if [ $n -eq 0 ]; then
        echo "Zero"
    else
        echo "Positive"
    fi
else
    echo "Negative"
fi
```

6.5 Multiple conditions

```
if [ $a -gt 0 ] && [ $a -lt 100 ]; then echo "between 1 and 99"; fi
if [[ $a -gt 0 && $a -lt 100 ]]; then echo "same, using [[ ]]"; fi
if [ $a -lt 0 ] || [ $a -gt 100 ]; then echo "out of range"; fi
if [ ! -f data.txt ]; then echo "data.txt missing"; fi
if (( a > 0 && a < 100 )); then echo "arithmetic style"; fi
```

6.6 Short-circuit (one-line) style

```
[ -f file.txt ] && echo "exists" || echo "not found"
mkdir backup 2>/dev/null || echo "backup already exists"
command1 && command2      # command2 runs only if command1 succeeded
command1 || command2     # command2 runs only if command1 failed
```

6.7 case statement (multi-way branch)

Syntax

```
case $variable in
    pattern1) commands ;;
    pattern2|pattern3) commands ;; # OR patterns
    *) default commands ;; # default
esac

read -p "Enter a character: " ch
case $ch in
    [aeiouAEIOU]) echo "Vowel" ;;
    [a-zA-Z]) echo "Consonant" ;;
    [0-9]) echo "Digit" ;;
    *) echo "Special character" ;;
esac

# menu example
echo "1) Date 2) Users 3) Disk"
read -p "Choice: " c
case $c in
    1) date ;;
    2) who ;;
    3) df -h ;;
    *) echo "Invalid choice" ;;
esac
```

Exam Tip: Each case branch must end with `;;`. `esac` (case reversed) closes it, just as `fi` closes `if`. `;;&` and `;&` (fall-through) exist only in bash 4+.

7. LOOPS (very important)

A loop repeats a block of commands. Bash has **for**, **while**, **until** and **select**. Loop control statements are **break** and **continue**.

Loop	Runs when / until	Use when
for	Once for each item in a list / until counter ends	Number of iterations is known
while	WHILE the condition is TRUE	Repeat until something becomes false
until	UNTIL the condition becomes TRUE (runs while it is FALSE)	Repeat until something becomes true
select	Repeats a numbered menu until break	Menu-driven scripts

7.1 for loop - list form

Syntax

```
for variable in item1 item2 item3 ...
do
    commands
done

for fruit in apple banana mango
do
    echo "I like $fruit"
done

for i in 1 2 3 4 5; do echo -n "$i "; done; echo
```

Output

```
I like apple
I like banana
I like mango
1 2 3 4 5
```

Ranges, steps and seq

```
for i in {1..5}; do echo -n "$i "; done; echo      # 1 2 3 4 5
for i in {10..1}; do echo -n "$i "; done; echo    # 10 9 8 ... 1
for i in {1..10..2}; do echo -n "$i "; done; echo # 1 3 5 7 9 (bash 4+ for step)
for i in $(seq 1 5); do echo -n "$i "; done; echo # works on macOS bash 3.2 too
for i in $(seq 0 5 20); do echo -n "$i "; done; echo # 0 5 10 15 20 (start step end)
for c in {a..e}; do echo -n "$c "; done; echo     # a b c d e
```

Exam Tip: Brace expansion `{1..$n}` does NOT work with variables. Use `seq 1 $n` or the C-style loop instead.

7.2 for loop - C-style (three-expression)

Syntax

```
for (( initialization; condition; increment ))
do
    commands
done

for (( i = 1; i <= 5; i++ ))
do
    echo "Iteration $i"
done

# sum of 1..10
sum=0
for (( i = 1; i <= 10; i++ )); do (( sum += i )); done
echo "Sum = $sum"

# countdown
for (( i = 5; i >= 1; i-- )); do echo -n "$i "; done; echo "Liftoff!"
```

Output

```
Iteration 1
Iteration 2
Iteration 3
Iteration 4
Iteration 5
Sum = 55
5 4 3 2 1 Liftoff!
```

7.3 for loop - over files, arguments and command output

```
for f in *.txt; do echo "File: $f"; done           # all .txt files
for f in /etc/*.conf; do [ -r "$f" ] && echo "$f"; done
for arg in "$@"; do echo "Arg: $arg"; done         # every command-line argument
for user in $(cut -d: -f1 /etc/passwd | head -3); do echo "$user"; done
for i in ${arr[@]}; do echo $i; done               # over an array
for ( ; ; ); do echo "infinite"; break; done       # infinite for (with break)
for dir in */; do echo "Directory: $dir"; done     # only directories
```

7.4 while loop

Executes the body **as long as the condition is true** (exit status 0). The condition is checked **BEFORE** each iteration, so the body may run zero times.

Syntax

```
while [ condition ]
do
    commands
    (update so the condition eventually becomes false)
done

i=1
while [ $i -le 5 ]
do
    echo "i = $i"
    (( i++ ))
done
```

```
# using (( )) in the condition
n=10; sum=0
while (( n > 0 )); do (( sum += n )); (( n-- )); done
echo "Sum = $sum"
```

Output

```
i = 1
i = 2
i = 3
i = 4
i = 5
Sum = 55
```

Reading a file line by line (very common)

```
while IFS= read -r line
do
    echo "Line: $line"
done < input.txt

# from a command
ls | while read -r name; do echo "Found $name"; done

# numbering lines
n=1
while read -r line; do echo "$n: $line"; ((n++)); done < input.txt

# split fields with IFS
while IFS=: read -r user x uid rest; do echo "$user -> $uid"; done < /etc/passwd
```

Infinite while loop

```
while true          # or   while :      or   while [ 1 ]
do
    read -p "Type quit to exit: " word
    [ "$word" = "quit" ] && break
done
echo "Bye"
```

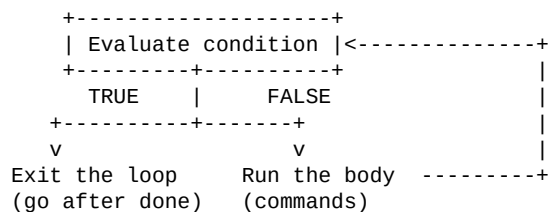
7.5 until loop (IMPORTANT - opposite of while)

The **until** loop repeats the commands **until the condition becomes TRUE**. In other words it runs **while the condition is FALSE** (non-zero exit status). The condition is checked before each iteration, and the loop stops as soon as the condition succeeds (exit status 0).

Syntax

```
until [ condition ]
do
    commands
    (change something so the condition eventually becomes true)
done
```

Flow of until loop



Example 1: print 1 to 5

```
i=1
until [ $i -gt 5 ]
do
    echo "Number: $i"
    (( i++ ))
done
```

Output

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

The loop runs while `i -gt 5` is FALSE (`i = 1..5`). When `i` becomes 6 the condition is TRUE, so it stops.

Example 2: countdown

```
count=5
until [ $count -eq 0 ]
do
    echo "T-minus $count"
    (( count-- ))
done
echo "Blast off!"
```

Output

```
T-minus 5
T-minus 4
T-minus 3
T-minus 2
T-minus 1
Blast off!
```

Example 3: sum of numbers 1 to n

```
read -p "Enter n: " n
i=1; sum=0
until (( i > n ))
do
    (( sum += i ))
    (( i++ ))
done
echo "Sum of 1 to $n = $sum"
```

Output

```
Enter n: 10
Sum of 1 to 10 = 55
```

Example 4: keep asking until valid input

```
answer=""
until [ "$answer" = "yes" ]
do
    read -p "Type 'yes' to continue: " answer
done
echo "Thank you!"
```

Example 5: guessing game with until

```
secret=7; guess=0; tries=0
until [ $guess -eq $secret ]
do
    read -p "Guess the number (1-10): " guess
    (( tries++ ))
    if [ $guess -lt $secret ]; then echo "Too low"
    elif [ $guess -gt $secret ]; then echo "Too high"
    fi
done
echo "Correct! You needed $tries tries."
```

Example 6: wait until a file appears

```
until [ -f /tmp/ready.txt ]
do
    echo "Waiting for /tmp/ready.txt ..."
    sleep 2
done
echo "File found, continuing."
```

Example 7: wait until a host is reachable / command succeeds

```
until ping -c 1 -W 1 8.8.8.8 > /dev/null 2>&1    # -W 1 on Linux; macOS uses -t 1
do
    echo "Network down, retrying in 3s..."
    sleep 3
done
echo "Network is up"

# retry a command up to 5 times
tries=0
until some_command || (( tries == 5 ))
do
    (( tries++ )); sleep 1
done
```

Example 8: factorial using until

```
read -p "Enter n: " n
f=1; i=1
until [ $i -gt $n ]
do
    f=$(( f * i ))
    (( i++ ))
done
echo "$n! = $f"
```

Output

```
Enter n: 5
5! = 120
```

Example 9: multiplication table with until

```
read -p "Table of: " n
i=1
until [ $i -gt 10 ]
do
    echo "$n x $i = $(( n * i ))"
    ((i++))
done
```

while versus until (same task written both ways)

```
# while: continue while i <= 5
i=1; while [ $i -le 5 ]; do echo $i; ((i++)); done

# until: continue until i > 5
i=1; until [ $i -gt 5 ]; do echo $i; ((i++)); done
```

while loop	until loop
Runs while condition is TRUE	Runs while condition is FALSE (until it becomes TRUE)
Stops when condition becomes false	Stops when condition becomes true
<code>while [\$i -le 5]</code>	<code>until [\$i -gt 5]</code> (negated condition)
Condition tested at the top (entry-controlled)	Condition tested at the top (entry-controlled)
May execute zero times	May execute zero times
Infinite form: <code>while true</code>	Infinite form: <code>until false</code>

Exam Tip: Bash has NO built-in do-while / repeat-until (test at the bottom). Unlike C's do-while, bash **until** checks first. To make the body run at least once, write: **while true; do body; [cond] && break; done.**

7.6 break, continue and exit

Statement	Effect
break	Leave the innermost loop immediately
break n	Leave n nested loops
continue	Skip rest of this iteration; go to next iteration
continue n	Continue with the next iteration of the n-th enclosing loop
exit [n]	Terminate the whole script with status n
return [n]	Leave a function with status n

```
# break: stop at 5
for i in {1..10}; do
    if [ $i -eq 5 ]; then break; fi
    echo -n "$i "
done; echo
# continue: skip 3
for i in {1..5}; do
    if [ $i -eq 3 ]; then continue; fi
    echo -n "$i "
done; echo
# skip even numbers
for i in {1..9}; do (( i % 2 == 0 )) && continue; echo -n "$i "; done; echo
```

Output

```
1 2 3 4
1 2 4 5
1 3 5 7 9
```

7.7 Nested loops

```
# Right triangle of stars
for (( i = 1; i <= 4; i++ ))
do
    for (( j = 1; j <= i; j++ )); do echo -n "* "; done
    echo
done

# break out of two loops
for i in 1 2 3; do
    for j in 1 2 3; do
        [ $j -eq 2 ] && [ $i -eq 2 ] && break 2
        echo "i=$i j=$j"
    done
done
```

Output

```
*
* *
* * *
* * * *
i=1 j=1
i=1 j=2
i=1 j=3
i=2 j=1
```

7.8 select loop (menu)

```
PS3="Choose an option: "      # prompt used by select
select opt in "Show date" "Show users" "Quit"
do
    case $opt in
        "Show date")  date ;;
        "Show users") who ;;
        "Quit")       break ;;
        *)            echo "Invalid: $REPLY" ;;
    esac
done
```

Output

```
1) Show date
2) Show users
3) Quit
Choose an option: 1
Wed Sep 30 10:15:02 +06 2026
```

Exam Tip: `select` shows a numbered list; the number typed is stored in `$REPLY` and the chosen text in the variable. It loops forever until `break` or EOF (Ctrl+D).

7.9 Loop tricks used in exams

```
# loop with sleep (animation / polling)
for i in {1..3}; do echo "tick $i"; sleep 1; done

# loop over lines of a command
who | while read user rest; do echo "User: $user"; done

# one-line loops
for i in {1..3}; do echo $i; done
while read l; do echo $l; done < file.txt

# do-while emulation
while true; do
    read -p "Number (0 stops): " n
    [ "$n" -eq 0 ] && break
    echo "You typed $n"
done

# loop with the classic 'i=$((i+1))'
i=0; while [ $i -lt 3 ]; do echo $i; i=$((i+1)); done
```

8. Arrays

8.1 Indexed Arrays

```
fruits=(apple banana mango)      # create
fruits[3]="orange"                # add by index
fruits+=("grape")                # append
echo ${fruits[0]}                 # first element -> apple
echo ${fruits[@]}                 # all elements
echo ${#fruits[@]}                # number of elements -> 5
echo ${!fruits[@]}                # all indexes -> 0 1 2 3 4
echo ${fruits[@]:1:2}             # slice: 2 elements from index 1 -> banana mango
echo ${#fruits[0]}                # length of first element -> 5
unset fruits[1]                   # delete element
unset fruits                      # delete entire array

for f in "${fruits[@]}"; do echo "Fruit: $f"; done      # loop over values
for i in "${!fruits[@]}"; do echo "$i -> ${fruits[$i]}"; done # loop over indexes
for (( i=0; i<${#fruits[@]}; i++ )); do echo ${fruits[$i]}; done
```

Sum and max of an array

```
nums=(4 9 2 7 5)
sum=0; max=${nums[0]}
for n in "${nums[@]"; do
    (( sum += n ))
    (( n > max )) && max=$n
done
echo "Sum=$sum Max=$max"
```

Output

```
Sum=27 Max=9
```

8.2 Associative Arrays [bash 4+]

```
declare -A marks          # must declare with -A
marks[Rahim]=85
marks[Karim]=72
marks=( [Rahim]=85 [Karim]=72 [Sumi]=91 )    # or assign together
echo ${marks[Karim]}      # 72
echo ${!marks[@]}         # keys
echo ${marks[@]}          # values
for k in "${!marks[@]"; do echo "$k has ${marks[$k]}"; done
```

9. Functions

Syntax (both forms are valid: `name() { }` and `function name { }`)

```
function_name() {
    commands
    return value          # optional, 0-255
}
function_name arg1 arg2  # calling (no parentheses!)

greet() {
    echo "Hello, $1! You are $2 years old."
}
greet "Hamim" 21

add() {
    local sum=$(( $1 + $2 ))    # local: visible only inside function
    echo $sum                  # "return" a value by printing
}
result=$(add 5 7)              # capture the output
echo "Sum = $result"

is_even() { (( $1 % 2 == 0 )); } # exit status becomes the return value
if is_even 10; then echo "even"; fi

check() { return 3; }
check; echo "Status: $?"
```

Output

```
Hello, Hamim! You are 21 years old.
Sum = 12
even
Status: 3
```

Recursion: factorial

```
fact() {
    if [ $1 -le 1 ]; then
        echo 1
    else
        local prev=$(fact $(( $1 - 1 )))
        echo $(( $1 * prev ))
    fi
}
echo "5! = $(fact 5)"
```

Output

```
5! = 120
```

Exam Tip: `return` can only give a number from 0 to 255 (a status). To return strings or big numbers, `echo` them and capture with `$( )`. Variables are GLOBAL by default; use `local` inside functions. Functions must be defined BEFORE they are called. Arguments inside a function are `$1` `$2` `##` `@$` of the function.

10. String Manipulation

```
s="Hello World"
echo ${#s}           # length -> 11
echo ${s:0:5}        # substring from 0, length 5 -> Hello
echo ${s:6}          # from index 6 to end -> World
echo ${s: -5}        # last 5 chars (space before -) -> World
echo ${s/World/Bash} # replace first match -> Hello Bash
echo ${s//o/o}        # replace ALL matches -> Hellø Wørlð
echo ${s#Hello }      # remove shortest prefix -> World
echo ${s%World}       # remove shortest suffix -> "Hello "
f="report.final.txt"
echo ${f%.*}          # report.final (remove extension)
echo ${f##*.}         # txt (longest prefix removed -> extension)
echo ${f%.*}          # report (longest suffix removed)
p="/home/user/file.txt"
echo ${p##*/}         # file.txt (same as basename)
echo ${p%/*}          # /home/user (same as dirname)
```

Expression	Meaning
<code>\${var:-default}</code>	Use default if var is unset or empty (var not changed)
<code>\${var:=default}</code>	Assign default to var if unset or empty, then use it
<code>\${var:?message}</code>	Print message and exit if var is unset or empty
<code>\${var:+alt}</code>	Use alt only if var IS set
<code>\${var^^} \${var,,}</code>	UPPERCASE / lowercase [bash 4+]
<code>\${var^}</code>	capitalise first letter [bash 4+]

```
echo "${name:-Guest}" # Guest (if name not set)
upper=$(echo "$s" | tr 'a-z' 'A-Z') # works on macOS bash 3.2 too
lower=$(echo "$s" | tr '[:upper:]' '[:lower:]')
rev=$(echo "$s" | rev) # dlrow olleH
a="Hello"; b="World"; c="$a $b" # concatenation
[ "$a" = "$b" ] && echo same || echo different
[[ $s == Hello* ]] && echo "starts with Hello"
[[ $s == *World ]] && echo "ends with World"
[[ "abc123" =~ ^[a-z]+[0-9]+$ ]] && echo "regex matched"
echo "a,b,c" | cut -d, -f2 # b
IFS=, read -ra parts <<< "a,b,c"; echo ${parts[1]} # b
```

11. Redirection, Pipes and Here Documents

Symbol	Meaning
<code>></code>	Redirect stdout to file (overwrite)
<code>>></code>	Append stdout to file
<code><</code>	Take stdin from file
<code>2></code>	Redirect stderr to file
<code>2>&1</code>	Send stderr to the same place as stdout
<code>&> or > f 2>&1</code>	Both stdout and stderr to file
<code>/dev/null</code>	Black hole: discards everything written to it

Symbol	Meaning
	Pipe: stdout of left command becomes stdin of right command
tee file	Write to file AND to the screen

File descriptors: **0** = stdin, **1** = stdout, **2** = stderr.

```
ls > list.txt           # overwrite
echo "more" >> list.txt # append
sort < list.txt         # input from file
ls /bad 2> errors.txt   # errors to file
ls /bad /etc > out.txt 2>&1 # both to the same file
command > /dev/null 2>&1 # silence everything
ls | wc -l             # count files
cat data.txt | sort | uniq -c | sort -nr | head -5 # pipeline
ls | tee files.txt      # show + save
```

```
# Here document: multi-line input
cat << EOF
Dear $USER,
Today is $(date +%A).
EOF
```

```
# Here string
tr 'a-z' 'A-Z' <<< "hello"
```

```
# write a file using a here-doc
cat > config.txt << 'EOF'
name=demo
mode=test
EOF
```

Output

```
Dear hamim,
Today is Wednesday.
HELLO
```

Exam Tip: `<< 'EOF'` (quoted delimiter) prevents variable expansion. `<<- EOF` strips leading tabs.

12. Command-line Arguments, shift, getopt

```
#!/bin/bash
if [ $# -lt 2 ]; then
    echo "Usage: $0 num1 num2"
    exit 1
fi
echo "Sum = $(( $1 + $2 ))"

# process all arguments using shift
while [ $# -gt 0 ]
do
    echo "Processing: $1"
    shift # drop $1, $2 becomes $1 ...
done
```

Output

```
$ ./sum.sh 4 6
Sum = 10
Processing: 4
Processing: 6
```

getopts (options such as -n -v -f file)

```
#!/bin/bash
verbose=0
while getopts "n:vh" opt
do
    case $opt in
        n) name=$OPTARG ;;          # option that takes a value (n:)
        v) verbose=1 ;;
        h) echo "Usage: $0 [-n name] [-v]"; exit 0 ;;
        \?) echo "Invalid option"; exit 1 ;;
    esac
done
echo "Name=$name Verbose=$verbose"
```

Output

```
$ ./opts.sh -n Hamim -v
Name=Hamim Verbose=1
```

13. Exit Status, trap, Debugging

13.1 Exit status

Every command returns a status in **\$?**. **0 = success**, non-zero = failure. Common codes: 1 general error, 2 misuse, 126 not executable, 127 command not found, 130 Ctrl+C.

```
grep -q "root" /etc/passwd
if [ $? -eq 0 ]; then echo "found"; else echo "not found"; fi
if grep -q "root" /etc/passwd; then echo "found"; fi    # better form
cd /some/dir || { echo "cannot cd"; exit 1; }
exit 0          # end script with success
```

13.2 set options

Command	Effect
<code>set -e</code>	Exit immediately if any command fails
<code>set -u</code>	Error when an unset variable is used
<code>set -x</code>	Print each command before executing (trace)
<code>set -o pipefail</code>	Pipeline fails if any command in it fails
<code>set +x</code>	Turn the option off
Safe header	<code>set -euo pipefail</code>

13.3 trap (signal handling)

```
cleanup() { echo "Cleaning up..."; rm -f /tmp/mytemp.$$; }
trap cleanup EXIT          # run when the script exits
trap 'echo "Ctrl+C ignored"' INT  # handle Ctrl+C (SIGINT)
trap - INT                 # reset to default
kill -l                    # list signals (SIGINT=2, SIGKILL=9, SIGTERM=15)
```

13.4 Debugging

```
bash -x script.sh          # trace every command
bash -n script.sh          # syntax check only, do not run
bash -v script.sh          # print lines as they are read
set -x ... set +x          # debug only a part of the script
echo "DEBUG: x=$x" >&2      # manual debug print
```

14. Useful Commands for Scripting

Command	Purpose	Example
grep	search text using patterns	grep -i -n "error" log.txt (-c count, -v invert, -r recursive)
sed	stream editor: substitute/delete	sed 's/old/new/g' f sed '2d' f sed -n '2,4p' f
awk	field-based processing	awk -F: '{print \$1}' /etc/passwd
cut	extract columns	cut -d, -f1,3 data.csv
sort	sort lines	sort -n -r file (-u unique, -k2 column)
uniq	remove adjacent duplicates	sort f uniq -c
wc	count	wc -l lines, -w words, -c bytes
head / tail	first / last lines	head -n 5 f tail -f log
tr	translate characters	tr a-z A-Z tr -d ' '
find	search files	find . -name "*.sh" -mtime -7
xargs	build command from stdin	find . -name '*.tmp' xargs rm
basename / dirname	split path	basename /a/b.txt -> b.txt
date	date and time	date +%Y-%m-%d date +%H:%M:%S
sleep	pause	sleep 2
bc	calculator (decimals)	echo "5/2" bc -l
expr	evaluate expression	expr 5 + 3 expr length "abc"

awk essentials

```
awk '{print $1, $3}' file           # columns 1 and 3
awk -F, '{sum += $2} END {print sum}' data.csv # total of column 2
awk 'NR > 1 && $3 > 50' marks.txt  # condition on lines
awk '{print NR ": " $0}' file      # number the lines
awk 'BEGIN{print "Start"} {n++} END{print n " lines"}' file
```

sed essentials

```
sed 's/cat/dog/' f                # replace first per line
sed 's/cat/dog/g' f                # replace all
sed -i.bak 's/a/b/g' f             # edit in place with backup (macOS needs -i '' or -i.bak)
sed '/^$/d' f                      # delete blank lines
sed -n '3p' f                      # print line 3 only
```

15. File Handling and Process Control

```
touch a.txt                       # create empty file
mkdir -p project/src              # create nested directories
cp a.txt b.txt mv b.txt c.txt rm c.txt rm -r project
cat a.txt less a.txt wc -l a.txt
ls -l                             # permissions: -rwxr-xr-- owner group others
chmod 755 f chmod u+x f chmod go-w f
chown user:group f
ln -s target linkname            # symbolic link
```

Digit	Permission	Digit	Permission
4	r (read)	7	rwx
2	w (write)	6	rw-
1	x (execute)	5	r-x

Process control

```
sleep 60 &                # run in background
echo $!                   # PID of background job
jobs                       # list background jobs
fg %1      bg %1          # bring to foreground / resume in background
ps aux | grep sleep       # list processes
kill 1234                  # ask process to stop (SIGTERM)
kill -9 1234              # force kill (SIGKILL)
nohup ./long.sh &         # keep running after logout
wait                       # wait for all background jobs to finish
crontab -e                 # schedule jobs: min hour day month weekday command
# 30 2 * * * /home/user/backup.sh    -> every day 02:30
```

16. Exam Programs (20 Solved Programs)

Each program is written to be short enough to reproduce in an exam. Practise writing them from memory.

Program 1: Even or odd

```
#!/bin/bash
read -p "Enter a number: " n
if [ $((n % 2)) -eq 0 ]; then
    echo "$n is Even"
else
    echo "$n is Odd"
fi
```

Program 2: Largest of three numbers

```
#!/bin/bash
read -p "Enter three numbers: " a b c
if [ $a -ge $b ] && [ $a -ge $c ]; then echo "Largest = $a"
elif [ $b -ge $c ]; then echo "Largest = $b"
else echo "Largest = $c"
fi
```

Program 3: Factorial (loop)

```
#!/bin/bash
read -p "Enter n: " n
fact=1
for (( i = 2; i <= n; i++ )); do fact=$(( fact * i )); done
echo "Factorial of $n = $fact"
```

Output

```
Enter n: 6
Factorial of 6 = 720
```

Program 4: Fibonacci series

```
#!/bin/bash
read -p "How many terms? " n
a=0; b=1
echo -n "Fibonacci: "
for (( i = 0; i < n; i++ ))
do
    echo -n "$a "
    t=$(( a + b )); a=$b; b=$t
done
echo
```

Output

```
How many terms? 8
Fibonacci: 0 1 1 2 3 5 8 13
```

Program 5: Prime number check

```
#!/bin/bash
read -p "Enter a number: " n
prime=1
if [ $n -lt 2 ]; then prime=0; fi
for (( i = 2; i * i <= n; i++ ))
do
    if [ $(n % i) -eq 0 ]; then prime=0; break; fi
done
[ $prime -eq 1 ] && echo "$n is Prime" || echo "$n is Not prime"
```

Program 6: Prime numbers up to N

```
#!/bin/bash
read -p "Upper limit: " N
for (( n = 2; n <= N; n++ ))
do
    p=1
    for (( i = 2; i * i <= n; i++ )); do (( n % i == 0 )) && { p=0; break; }; done
    (( p )) && echo -n "$n "
done
echo
```

Output

```
Upper limit: 30
2 3 5 7 11 13 17 19 23 29
```

Program 7: Palindrome (string)

```
#!/bin/bash
read -p "Enter a word: " w
r=$(echo "$w" | rev)
if [ "$w" = "$r" ]; then echo "$w is a palindrome"
else echo "$w is not a palindrome"; fi
```

Program 8: Reverse a number and sum of digits

```
#!/bin/bash
read -p "Enter a number: " n
orig=$n; rev=0; sum=0
while [ $n -gt 0 ]
do
    d=$(( n % 10 ))
    rev=$(( rev * 10 + d ))
    sum=$(( sum + d ))
    n=$(( n / 10 ))
done
echo "Reverse = $rev, Sum of digits = $sum"
[ $orig -eq $rev ] && echo "Palindrome number"
```

Output

```
Enter a number: 1234
Reverse = 4321, Sum of digits = 10
```

Program 9: Armstrong number

```
#!/bin/bash
read -p "Enter a number: " n
len=${#n}; t=$n; sum=0
while [ $t -gt 0 ]
do
    d=$(( t % 10 ))
    sum=$(( sum + d ** len ))
    t=$(( t / 10 ))
done
[ $sum -eq $n ] && echo "$n is Armstrong" || echo "$n is not Armstrong"
```

Output

```
Enter a number: 153
153 is Armstrong
```

Program 10: Multiplication table

```
#!/bin/bash
read -p "Enter a number: " n
for i in {1..10}; do printf "%d x %2d = %3d\n" $n $i $((n * i)); done
```

Program 11: Leap year

```
#!/bin/bash
read -p "Enter year: " y
if (( (y % 4 == 0 && y % 100 != 0) || y % 400 == 0 )); then
    echo "$y is a leap year"
else
    echo "$y is not a leap year"
fi
```

Program 12: Count vowels in a string

```
#!/bin/bash
read -p "Enter a string: " s
count=0
for (( i = 0; i < ${#s}; i++ ))
do
    case "${s:i:1}" in [aeiouAEIOU]) (( count++ )) ;; esac
done
echo "Vowels = $count"
# one-liner: echo "$s" | grep -o -i '[aeiou]' | wc -l
```

Output

```
Enter a string: Education
Vowels = 5
```

Program 13: Menu-driven calculator (with while + case)

```
#!/bin/bash
while true
do
    echo "1.Add 2.Sub 3.Mul 4.Div 5.Exit"
    read -p "Choice: " ch
    [ "$ch" -eq 5 ] && { echo "Bye"; break; }
    read -p "Enter two numbers: " a b
    case $ch in
        1) echo "Result = $(( a + b ))" ;;
        2) echo "Result = $(( a - b ))" ;;
        3) echo "Result = $(( a * b ))" ;;
        4) if [ $b -eq 0 ]; then echo "Cannot divide by zero"
            else echo "Result = $(echo "scale=2; $a/$b" | bc)"; fi ;;
        *) echo "Invalid choice" ;;
    esac
done
```

Program 14: File / directory checker

```
#!/bin/bash
read -p "Enter path: " p
if [ -d "$p" ]; then echo "$p is a directory"
elif [ -f "$p" ]; then
    echo "$p is a regular file"
    [ -r "$p" ] && echo "readable"
    [ -w "$p" ] && echo "writable"
    [ -x "$p" ] && echo "executable"
    echo "Lines: $(wc -l < "$p")"
else echo "$p does not exist"
fi
```

Program 15: Grade calculator using case on ranges

```
#!/bin/bash
read -p "Enter marks (0-100): " m
case $(( m / 10 )) in
    10|9|8) g="A+" ;;
    7)      g="A"  ;;
    6)      g="B"  ;;
    5)      g="C"  ;;
    *)      g="F"  ;;
esac
echo "Grade: $g"
```

Program 16: Temperature conversion (decimals with bc)

```
#!/bin/bash
read -p "Celsius: " c
f=$(echo "scale=2; $c * 9 / 5 + 32" | bc)
echo "$c C = $f F"
```

Output

```
Celsius: 37
37 C = 98.60 F
```

Program 17: Pattern printing (pyramid)

```
#!/bin/bash
read -p "Rows: " n
for (( i = 1; i <= n; i++ ))
do
    for (( s = 1; s <= n - i; s++ )); do echo -n " "; done
    for (( j = 1; j <= 2 * i - 1; j++ )); do echo -n " "; done
    echo
done
```

Output

```
Rows: 4
*
***
*****
*****
```

Program 18: Number guessing game using until

```
#!/bin/bash
secret=$(( RANDOM % 10 + 1 ))    # random 1-10
guess=0; tries=0
until [ $guess -eq $secret ]
do
    read -p "Guess (1-10): " guess
    (( tries++ ))
    [ $guess -lt $secret ] && echo "Too low"
    [ $guess -gt $secret ] && echo "Too high"
done
echo "You got it in $tries tries!"
```

Program 19: Backup files of a directory with timestamp

```
#!/bin/bash
src=${1:-.}                                # default = current directory
dest="backup_$(date +%Y%m%d_%H%M%S)"
mkdir -p "$dest" || exit 1
count=0
for f in "$src"/*
do
    if [ -f "$f" ]; then cp "$f" "$dest"/; (( count++ )); fi
done
echo "$count files copied to $dest"
```

Program 20: Word frequency / count lines, words, chars of a file

```
#!/bin/bash
[ $# -ne 1 ] && { echo "Usage: $0 file"; exit 1; }
[ -f "$1" ] || { echo "No such file"; exit 2; }
echo "Lines : $(wc -l < "$1")"
echo "Words : $(wc -w < "$1")"
echo "Chars : $(wc -c < "$1")"
echo "Top words:"
tr -s ' ' '\n' < "$1" | tr 'A-Z' 'a-z' | sort | uniq -c | sort -nr | head -3
```

17. Viva / Short Questions and Answers

No.	Question	Answer
1	What is a shell?	A command interpreter that provides an interface between the user and the kernel.
2	Difference between shell and kernel?	Kernel is the core of the OS that manages hardware; shell takes user commands and passes them to the kernel.
3	What is the shebang?	The first line <code>#!/bin/bash</code> that specifies the interpreter used to run the script.
4	How do you make a script executable?	<code>chmod +x script.sh</code> and run it with <code>./script.sh</code> .
5	**Difference between <code>\$*</code> and <code>\$@</code> ?*	Quoted <code>"\$"</code> is one single string; quoted <code>"\$@"</code> keeps each argument as a separate word.
6	What does <code>\$?</code> hold?	Exit status of the last executed command; 0 means success.
7	What is <code>\$\$</code> ?	The process ID of the current shell or script.
8	Difference between <code>=</code> and <code>-eq</code> ?	<code>=</code> compares strings; <code>-eq</code> compares integers.
9	Difference between <code>while</code> and <code>until</code> ?	<code>while</code> runs while the condition is true; <code>until</code> runs while the condition is false (stops when it becomes true).
10	Difference between <code>break</code> and <code>continue</code> ?	<code>break</code> exits the loop entirely; <code>continue</code> skips the rest of the current iteration.
11	What is <code>\$(())</code> ?	Arithmetic expansion - evaluates an integer expression.
12	How to do floating point maths?	Use <code>bc</code> or <code>awk</code> , because bash arithmetic supports integers only.
13	What is <code>/dev/null</code> ?	A special file that discards anything written to it.
14	Difference between <code>></code> and <code>>></code> ?	<code>></code> overwrites the file; <code>>></code> appends to it.
15	Difference between single and double quotes?	Double quotes allow variable/command expansion; single quotes keep everything literal.
16	What is <code>export</code> ?	Makes a variable available to child processes (environment variable).
17	Difference between <code>source</code> and <code>./script</code> ?	<code>source</code> runs in the current shell; <code>./script</code> runs in a new child shell.
18	What is a pipe?	Connects stdout of one command to stdin of another: <code>ls wc -l</code> .
19	What does <code>set -e</code> do?	Exits the script as soon as any command returns non-zero.
20	How to read a file line by line?	<code>while IFS= read -r line; do ...; done < file</code>
21	How are functions returned values?	<code>return</code> gives a status 0-255; use <code>echo</code> and <code>\$()</code> to return actual data.
22	What is <code>IFS</code> ?	Internal Field Separator - characters used for word splitting (default: space, tab, newline).
23	What is <code>trap</code> ?	Catches signals (INT, TERM, EXIT) and runs a handler command.
24	What is a here document?	<code><< EOF ... EOF</code> feeds multiple lines as stdin to a command.
25	Name the loops in bash.	<code>for</code> , <code>while</code> , <code>until</code> , <code>select</code> .
26	What are the standard file descriptors?	0 = stdin, 1 = stdout, 2 = stderr.
27	How to debug a script?	<code>bash -x script.sh</code> , <code>set -x</code> , or <code>bash -n</code> for syntax check.

No.	Question	Answer
28	What is the difference between <code>[]</code> and <code>[[]]</code> ?	<code>[[]]</code> is a bash extension that supports <code>&&</code> , <code> </code> , pattern and regex matching and does not need quoting.

18. Quick Cheat Sheet and macOS Notes

Task	Syntax
Shebang	<code>#!/bin/bash</code>
Variable	<code>x=5 echo \$x \${x}</code>
Read input	<code>read -p "msg: " x</code>
Math	<code>\$((a+b)) ((a++)) bc</code> for decimals
if	<code>if [\$a -gt \$b]; then ... elif ...; else ...; fi</code>
case	<code>case \$x in a) ...;; b c) ...;; *) ...;; esac</code>
for (list)	<code>for i in 1 2 3; do ...; done</code>
for (C-style)	<code>for ((i=0; i<5; i++)); do ...; done</code>
while	<code>while [\$i -le 5]; do ...; done</code>
until	<code>until [\$i -gt 5]; do ...; done</code>
Array	<code>a=(1 2 3) \${a[0]} \${a[@]} \${#a[@]}</code>
Function	<code>f() { echo \$1; } f hello</code>
Length of string	<code>\${#s}</code>
Command substitution	<code>x=\$(date)</code>
Last status	<code>\$?</code>
Args	<code>\$0 \$1 \$# \$@ \$*</code>
Redirect	<code>> >> < 2> 2>&1 </code>
Debug	<code>bash -x script.sh</code>

Test operator memory table

Numbers	Strings	Files
<code>-eq -ne -gt -lt -ge -le</code>	<code>= != -z -n</code>	<code>-e -f -d -r -w -x -s -L</code>

macOS-specific notes (you use macOS)

- Default shell is **zsh**. Run **bash** explicitly, or use `#!/bin/bash` as the shebang.
- System bash is **3.2**: no associative arrays (`declare -A`), no `${var^^}`, no `{1..10..2}` step, no `mapfile`. Install a newer one with `brew install bash` (usually `/opt/homebrew/bin/bash`), then use `#!/usr/bin/env bash`.
- macOS commands are BSD versions: `sed -i ''` (needs an empty backup extension), `date -v+1d` instead of `date -d tomorrow`, `ping -t` instead of `-W`, `readlink` lacks `-f`, `stat -f` instead of `stat -c`.
- `seq`, `bc`, `awk`, `tr`, `rev`, `cut` all work on macOS.
- Make executable with `chmod +x file.sh` and run `./file.sh`. In zsh, arrays start at index 1 but in bash they start at 0.

Common mistakes (lose marks here)

- Spaces around `=` in assignment: `x = 5` is an error.
- Missing spaces inside brackets: `[$a -eq 1]` is an error; write `[ $a -eq 1 ]`.
- Using `=` for numbers or `-eq` for strings.
- Forgetting `then` / `do` / `done` / `fi` / `esac` / `;;`.
- Forgetting quotes around variables that may contain spaces or be empty: use `"$var"`.
- Using `>` inside `[ ]` for numbers (it redirects to a file!). Use `-gt` or `(( ))`.
- Not updating the loop variable in `while/until` (creates an infinite loop).
- Expecting bash to do decimals: `echo $( (5/2) )` gives 2, not 2.5.
- Using `$` when assigning: `$x=5` is wrong.

Good luck with your exam!